PDF Smoke

Generated on: 2026-07-17T17:17:15.343Z

Table of Contents

- Architecture
- Examples
- Index
- Operations
- Report Schema

Architecture

Source: architecture.md

Architecture

Overview

docs-unifier is a TypeScript CLI that ingests documentation from one or more sources and emits:

- one consolidated Markdown document
- one consolidated PDF document (unless skipped)

Primary flow:

1. Parse CLI arguments into a ConsolidationInput model.
2. Resolve one or more source adapters.
3. Fetch raw pages from each source.
4. Normalize content to Markdown and harden links/assets.
5. Assemble one merged Markdown document with TOC.
6. Optionally render PDF.
7. Emit structured run metadata and optional JSON report.

Source Adapters

Supported sources:

- GitHub Wiki adapter
- Local filesystem adapter
- ReadTheDocs adapter

Each adapter returns a SourceFetchResult containing:

- pages
- adapter id

- source cache status
- optional detail message
- optional warning details

Caching Model

Two caching layers are used:

1. Source fetch cache

- Stores adapter-level raw page payloads.
- Supports TTL.
- Can return stale fallback for some remote sources.

2. Consolidation output cache

- Stores output metadata keyed by source digest and run options.
- Skips regeneration when source digest is unchanged.

Multi-Source Merge

When multiple sources are provided:

1. Sources are fetched in CLI argument order.
2. Each source's pages are offset in ordering to preserve source grouping.
3. Pages are concatenated and normalized into one merged output.

Run result includes:

- sourceFetch (primary/legacy)
- sourceFetches (full per-source list)

Normalization

Normalization pipeline:

1. Convert inputs to Markdown (Markdown passthrough, HTML conversion, RST conversion).
2. Clean markdown whitespace and structure.
3. Harden links:

- normalize relative document links
- rewrite relative asset/image links to ./assets/... virtual paths

4. Attach warnings when fallback behavior occurs.

Rendering

Markdown is assembled into a unified document with:

- title
- generated timestamp

- table of contents
- per-page sections and source references

PDF generation uses wkhtmltopdf over generated HTML.

Errors and Strict Mode

Strict stale fallback mode can fail a run when stale source cache fallback is used.

- Exit code: 2
- Report status: failed-strict-stale-fallback

Core Files

- src/cli.ts
- src/consolidate.ts
- src/adapters/
- src/pipeline/
- src/cache/
- report.schema.json

Examples

Source: examples.md

Examples and Expected Output

This page provides practical command examples with representative output snippets.

Example 1: Successful Local Run

Command:

```
node dist/cli.js --title "Docs" --local .\docs --skip-pdf --report-json .\out\run-success.jso
```

Representative console output:

```
Done. Pages: 42
Cache: miss
Source fetch: adapter=local cacheStatus=miss detail=Source pages read from filesystem and cac
Source cache stats: hits=0 misses=1 staleFallbacks=0 disabled=0
Markdown: C:\repo\out\combined.md
Warnings: 1
Report JSON: C:\repo\out\run-success.json
```

Representative report fields:


```
{
  "schemaVersion": "1.0.0",
  "status": "success",
  "result": {
    "sourceFetch": {
      "adapter": "local",
      "cacheStatus": "miss"
    },
    "sourceCacheStats": {
      "hits": 0,
      "misses": 1,
      "staleFallbacks": 0,
      "disabled": 0
    }
  }
}
```

Example 2: Multi-Source Success

Command:

```
node dist/cli.js --title "All Docs" --source local=.\docs --source rtd=https://project.readth
```

Representative console output:

```
Done. Pages: 128
Cache: miss
Source fetch: adapter=local cacheStatus=hit detail=Source pages loaded from local adapter cac
Source fetches:
- adapter=local cacheStatus=hit detail=Source pages loaded from local adapter cache.
- adapter=readthedocs cacheStatus=miss detail=ReadTheDocs pages crawled from network and cach
Source cache stats: hits=1 misses=1 staleFallbacks=0 disabled=0
Markdown: C:\repo\out\all.md
Report JSON: C:\repo\out\run-multi.json
```

Representative report fields:

```
{
  "status": "success",
  "input": {
    "sources": [
      { "sourceKind": "local", "sourceValue": ".\\docs" },
      { "sourceKind": "readthedocs", "sourceValue": "https://project.readthedocs.io/en/latest
    ]
  },
  "result": {
    "sourceFetches": [
      { "adapter": "local", "cacheStatus": "hit" },
      { "adapter": "readthedocs", "cacheStatus": "miss" }
    ]
  }
}
```

Example 3: Strict Stale Fallback Failure

Command:

```
node dist/cli.js --title "Docs" --rtd "https://project.readthedocs.io/en/latest/" --strict-st
```

Representative console output when stale fallback is used:

```
Done. Pages: 15
Cache: miss
Source fetch: adapter=readthedocs cacheStatus=stale-fallback detail=Network crawl produced no
Source cache stats: hits=0 misses=0 staleFallbacks=1 disabled=0
Markdown: C:\repo\out\combined.md
Warnings: 2
Report JSON: C:\repo\out\run-strict.json
Error: strict stale fallback mode is enabled and stale cached source data was used.
```

Outcome:

- Process exit code is 2.
- Report status is failed-strict-stale-fallback.

Representative report fields:

```
{
  "status": "failed-strict-stale-fallback",
  "result": {
    "warningDetails": [
      { "code": "SOURCE_STALE_FALLBACK_READTHEDOCS", "severity": "warning" },
      { "code": "SOURCE_STALE_FALLBACK", "severity": "warning" }
    ],
    "sourceFetch": {
      "adapter": "readthedocs",
      "cacheStatus": "stale-fallback"
    },
    "sourceCacheStats": {
      "staleFallbacks": 1
    }
  }
}
```

Example 4: Common Warning Interpretation

If pandoc is not installed and .rst files are processed, a warning can appear:

```
Pandoc not found or failed; .rst content embedded as raw block.
```

Mapped warning code:

- RST_PANDOC_UNAVAILABLE

If a relative image link is rewritten to virtual assets path, warning can appear:

```
Asset link rewritten to virtual assets path: ../images/logo.png -> ./assets/images/logo.png
```

Mapped warning code:

- ASSET_LINK_REWRITTEN

Index

Source: index.md

Documentation Index

- Architecture: docs/architecture.md
- Report Schema Guide: docs/report-schema.md
- Operations Runbook: docs/operations.md
- Examples and Expected Output: docs/examples.md

Operations

Source: operations.md

Operations Runbook

Prerequisites

Runtime requirements:

- Node.js 20+
- git in PATH (GitHub Wiki sources)
- wkhtmltopdf in PATH (PDF output)
- pandoc optional (better RST conversion)

Common Commands

For representative console/report outputs, see docs/examples.md.

Build:

```
npm run build
```

Run tests:

```
npm run test
```

Run report schema test only:

```
npm run test:report-schema
```

Typical Runs

Single local source:

```
node dist/cli.js --title "Docs" --local .\docs --out-md .\out\combined.md --out-pdf .\out\com
```

Single ReadTheDocs source:

```
node dist/cli.js --title "RTD" --rtd "https://project.readthedocs.io/en/latest/" --skip-pdf -
```

Multi-source merge:

```
node dist/cli.js --title "All Docs" --source local=.\docs --source rtd=https://project.readth
```

Caching Controls

Use default cache:

```
node dist/cli.js --title "Docs" --local .\docs --cache-dir .\.cache
```

Disable cache:

```
node dist/cli.js --title "Docs" --local .\docs --no-cache
```

Change source cache TTL:

```
node dist/cli.js --title "Docs" --local .\docs --source-cache-ttl-minutes 30
```

Strict Stale Fallback

Enable strict mode:

```
node dist/cli.js --title "Docs" --rtd "https://project.readthedocs.io/en/latest/" --strict-st
```

Behavior:

- If stale source fallback is used, process exits with code 2.
- Report status is failed-strict-stale-fallback.

Report Output

Generate report:

```
node dist/cli.js --title "Docs" --local .\docs --skip-pdf --report-json .\out\run-report.json
```

Validate report schema in CI:

- report.schema.json
- npm run test:report-schema

Troubleshooting

Issue: git clone failures for wiki sources

- Verify repository/wiki URL.
- Verify git is installed and in PATH.
- Check network/authentication.

Issue: PDF generation fails

- Ensure wkhtmltopdf is installed.
- Use --skip-pdf for markdown-only runs.

Issue: RST warnings present

- Install pandoc.
- Confirm pandoc is reachable in PATH.

Issue: strict mode failures

- Inspect warningDetails and sourceFetches in report.
- Restore source connectivity or run without strict mode.

Monitoring Signals

Watch in report/result:

- sourceCacheStats.staleFallbacks > 0
- warningDetails entries with SOURCE_STALE_FALLBACK*
- repeated RST_PANDOC_UNAVAILABLE

Report Schema

Source: report-schema.md

Report Schema Guide

Purpose

The run report is an optional JSON artifact produced via --report-json. It is intended for CI pipelines, observability, and automated quality checks.

Schema file:

- report.schema.json

Current schema version:

- 1.0.0

Top-Level Fields

Required top-level fields:

- schemaVersion
- generatedAt
- status
- input
- result

status values:

- success
- failed-strict-stale-fallback

Input Section

The input object captures effective runtime configuration:

- sourceKind
- sourceValue
- sources (for multi-source runs)
- title
- outMarkdownPath
- outPdfPath
- maxPages
- maxDepth
- skipPdf
- strictStaleFallback
- useCache
- cacheDir
- sourceCacheTtlMinutes

Result Section

The result object captures outcome details:

- markdownPath
- pdfPath (optional)
- pagesProcessed
- warnings (string messages)
- warningDetails (structured warning entries)
- fromCache
- sourceFetch (primary/legacy)
- sourceFetches (per-source list)
- sourceCacheStats

Warning Details

Each warning entry contains:

- code
- message
- severity (warning | error)

Example warning codes:

- RST_PANDOC_UNAVAILABLE
- ASSET_LINK_REWRITTEN
- SOURCE_STALE_FALLBACK
- SOURCE_STALE_FALLBACK_READTHEDOCS
- SOURCE_STALE_FALLBACK_GITHUB_WIKI

Source Cache Stats

sourceCacheStats includes:

- hits
- misses
- staleFallbacks
- disabled

These counters are aggregated across all sources in multi-source runs.

Validation in CI

Schema validation test command:

```
npm run test:report-schema
```

Full test suite command:

```
npm run test
```

Compatibility Guidance

When changing report payload shape:

1. Bump schemaVersion.
2. Update report.schema.json.
3. Update tests/reportSchema.test.ts.
4. Document migration notes in changelog/release notes.
